

H1 Report

- Name: 蔡芯惠
 - ID: D1149431
-

題目：象棋翻棋遊戲

設計方法概述

- Chess :
 - 封裝棋子的屬性：名稱、權重階級、陣營(0：紅方、1：黑方)、位置、是否翻開(設計了只能吃明棋的規則)。
- ChessGame :
 - 棋盤初始化(打亂棋子初始位置)
 - 移動合法性判定
 - 吃子規則(本遊戲設計砲/炮可藉由隔一棋無視棋子階級吃敵方任何棋子)。
- Main :
 - 使用者輸入與遊戲流程控制。

程式、執行畫面及其說明

AbstractGame 程式如下：

```
package org.example;

abstract class AbstractGame {
    public abstract void setPlayers(String Player1, String Player2);
    public abstract boolean gameOver();
    public abstract boolean move(int now_location, int target_location);
}
```

Chess 程式如下：

新增 isOpened 屬性，判斷選擇的棋子是否已經翻開。

```

package org.example;

class Chess {
    String name;
    int weight;
    int side;
    int location;
    boolean isOpened = false;

    Chess(String name, int weight, int side, int location) {
        this.name = name;
        this.weight = weight;
        this.side = side;
        this.location = location;
    }

    public String toString() {
        return isOpened ? " " + name : " X ";
    }
}

```

ChessGame 程式如下：

```

package org.example;

import java.util.ArrayList;
import java.util.Collections;

class ChessGame extends AbstractGame {
    Chess[] board = new Chess[32];
    int Player1_side = -1;
    int turn = 0;
}

```

setPlayer 程式如下：

```

public void setPlayers(String Player1, String Player2) {
    System.out.println("遊戲開始！" + '\n' + Player1 + " vs " + Player2);
}

```

gameOver 程式如下：

只要把另一方的棋子吃完即可獲勝。

```
public boolean gameOver() {
    int red = 0;
    int black = 0;

    for (Chess chess : board) {
        if (chess.side == 0) red++;
        else if (chess.side == 1) black++;
    }

    if (red == 0) {
        System.out.println("遊戲結束！黑方獲勝！");
        return true;
    }

    if (black == 0) {
        System.out.println("遊戲結束！紅方獲勝！");
        return true;
    }

    return false;
}
```

move 程式如下：

處理吃棋子後，原本位置應該要變空格位置，在原位置新創一個空地物件。

```
public boolean move(int now_location, int target_location) {
    if (!board[target_location].name.equals("__")) System.out.println(((board[now_location].side == 0) ? "(紅方)" : "(黑方)"));
    board[target_location] = board[now_location];
    board[now_location] = new Chess("__", 0, -1, now_location);
    board[now_location].isOpened = true; // 因為new， isOpened != true就會被顯示 X
    return true;
}
```

showAllChess 程式如下：

顯示當前棋局。

```

public void showAllChess() {
    char row = 'A';
    System.out.println("\t 1\t 2\t 3\t 4\t 5\t 6\t 7\t 8");
    for (int i = 0; i < 4; i++) {
        System.out.print(row + "\t");
        for (int j = 0; j < 8; j++) {
            System.out.print(board[j+i*8] + "\t");
        }
        System.out.println();
        row += 1;
    }
    System.out.println();
}

```

generateChess 程式如下：

創建32個棋子物件，加入到陣列中，透過函式將陣列順序打亂。

```

public void generateChess() {
    ArrayList<Chess> allList = new ArrayList<>();

    // 紅方 (side = 0)
    allList.add(new Chess("帥", 7, 0, -1));
    for (int i = 0; i < 2; i++) {
        allList.add(new Chess("仕", 6, 0, -1));
        allList.add(new Chess("相", 5, 0, -1));
        allList.add(new Chess("俥", 4, 0, -1));
        allList.add(new Chess("傌", 3, 0, -1));
        allList.add(new Chess("炮", 2, 0, -1));
    }

    for (int i = 0; i < 5; i++) {
        allList.add(new Chess("兵", 1, 0, -1));
    }

    // 黑方 (side = 1)
    allList.add(new Chess("將", 7, 1, -1));
    for (int i = 0; i < 2; i++) {
        allList.add(new Chess("士", 6, 1, -1));
        allList.add(new Chess("象", 5, 1, -1));
    }
}

```

```

    allList.add(new Chess("車", 4, 1, -1));
    allList.add(new Chess("馬", 3, 1, -1));
    allList.add(new Chess("砲", 2, 1, -1));
}

for (int i = 0; i < 5; i++) {
    allList.add(new Chess("卒", 1, 1, -1));
}

Collections.shuffle(allList);

for (int i = 0; i < 32; i++) {
    board[i] = allList.get(i);
    board[i].location = i;
}
}

```

countBetweenChess 程式如下：

本遊戲設置砲/炮只能移動至相鄰空格位置，如要吃子，必須恰好隔一個棋子才可以吃子。透過這個方法計算砲/炮與欲吃的子之間的棋子數，藉此判斷吃子行為是否合法。

```

// 選擇砲/炮吃棋子，中間需隔一個棋子
public int countBetweenChess(int now_location, int target_location) {
    int chessCount = 0;
    int now_row = now_location / 8, now_col = now_location % 8;
    int target_row = target_location / 8, target_col = target_location % 8;

    if (now_col == target_col) { // 上、下跳
        int min_row = Math.min(now_row, target_row);
        int max_row = Math.max(now_row, target_row);

        for (int i = min_row + 1; i < max_row; i++) {
            if (!board[8 * i + now_col].name.equals("__")) chessCount++;
        }
    }
    else if (now_row == target_row) { // 左、右跳
        int min_col = Math.min(now_col, target_col);
        int max_col = Math.max(now_col, target_col);

        for (int i = min_col + 1; i < max_col; i++) {
            if (!board[now_row * 8 + i].name.equals("__")) chessCount++;
        }
    }
}

```

```

    }
}
return chessCount;
}

```

nextTurn 程式如下： 切換回合之作用。

```

// 切換回合
public void nextTurn() {
    turn = (turn == 0) ? 1 : 0;
}

```

eat 程式如下：

處理吃子的行為，在主程式已通過一定的選擇規則，此方法中根據攻擊方和被攻擊方的權重階級，判斷攻擊方是否能吃掉被攻擊方。

另外寫了砲/炮的吃子及移動規則。

```

public boolean eat(int now_location, int target_location ) {
    Chess attacker = board[now_location];
    Chess target = board[target_location];
    boolean canEat = false;

    if (attacker.weight >= target.weight) canEat = true;
    else if (attacker.weight == 1 && target.weight == 7) canEat = true;
    else if (attacker.weight == 7 && target.weight == 1) canEat = false;
    else canEat = false;

    if (attacker.name.equals("砲") || attacker.name.equals("炮")) {
        int dist = Math.abs(now_location / 8 - target_location / 8) + Math.abs(now_location % 8 - target_location % 8);

        if (board[target_location].name.equals("__")) {
            if (dist == 1) {
                return move(now_location, target_location);
            }
        }
        else {
            System.out.println("只能移動至相鄰空格位置！");
            return false;
        }
    }
    else {

```

```

        int countChess = countBetweenChess(now_location, target_location);

        if (target.isOpened) {
            if (countChess == 1 && attacker.side != target.side) {
                return move(now_location, target_location);
            } else {
                System.out.println("無法吃我方棋子！");
                return false;
            }
        }
        else {
            System.out.println("無法吃蓋著的棋子！");
            return false;
        }
    }
}
else if (canEat) {
    return move(now_location, target_location);
}
else {
    System.out.println("無法吃掉對方棋子");
    return false;
}
}
}

```

```

// 確認欲吃掉的棋子是否為我方
public boolean checkSameSide(int origin_loc, int target_loc) {

    if (board[target_loc].name.equals("__")) {
        return true;
    }

    if (board[origin_loc].side == board[target_loc].side) { // 無效目的位置
        System.out.println("無法吃掉我方棋子！");
        return false;
    }
    else return true; // 有效目的位置，可移動
}

```

caltarget 程式如下：

計算目標位置在陣列中的位置。

```
// 將輸入的目標移動位置轉成整數型態
public int caltarget(String location) {
    int row = location.charAt(0) - 'A';
    int col = location.charAt(1) - '1';
    int index = row * 8 + col;

    return index;
}
```

colorSide 程式如下：

提醒使用者目前輪到哪一方。

```
public String colorSide() {
    String color = " ";
    if (Player1_side == 0) color = "(紅方)";
    else if (Player1_side == 1) color = "(黑方)";
    else color = " ";

    return color;
}
}
```

Main 程式如下：

負責遊戲流程以及少量棋子移動、選擇規則。

```
package org.example;

import java.util.Scanner;

public class Main {
    static void main(String[] args) {
        ChessGame game = new ChessGame();
        game.generateChess();
        game.setPlayers("小A", "小B");

        Scanner input = new Scanner(System.in);
```



```

while (!game.gameOver()) {
    int quit = 0;
    System.out.println();
    game.showAllChess();
    String currentPlayer = (game.turn == 0) ? "小A" : "小B";
    System.out.println("現在輪到: " + currentPlayer + " " + game.colorSide());
    System.out.print("請輸入欲翻開/移動之棋子位置(如：A1): ");

    String str_location = input.next().toUpperCase(); // 輸入位置
    int target_location = game.caltarget(str_location); // 取得整數型態的位置

    if (target_location < 0 || target_location > 31) {
        System.out.println("請輸入正確位置！");
        continue;
    }

    if (game.board[target_location].isOpened) { // 選擇已翻開棋子進行移動

        if (game.Player1_side != -1 && game.board[target_location].side != game.Player1_side) {
            System.out.println("此棋子並非你的，請重新選擇！");
            continue;
        }

        System.out.println("選擇之棋子為: " + game.board[target_location].name);

        while (true) {
            int now_location = target_location;
            quit = 0;

            System.out.print("請輸入目標移動位置(或輸入 Q 取消移動/吃子，改為翻開棋子): ");
            String sec_str_location = input.next().toUpperCase();

            if (sec_str_location.equals("Q")) {
                quit = 1;
                break;
            }
            else quit = 0;

            int sec_target_location = game.caltarget(sec_str_location);

            if (sec_target_location < 0 || sec_target_location > 31) {
                System.out.println("請輸入正確位置！");
                continue;
            }
        }
    }
}

```

```

    }

    int now_row = now_location / 8, now_col = now_location % 8;
    int target_row = sec_target_location / 8, target_col = sec_target_location % 8;
    int dist = Math.abs(now_row - target_row) + Math.abs(now_col - target_col);

    if (game.board[now_location].name.equals("砲") || game.board[now_location].name.equals("炮")) {
        if (dist == 1 && game.board[sec_target_location].side != -1) {
            System.out.println("砲/炮必須隔一個棋子才能吃子或只能移動至相鄰空格位置！");
            continue;
        }
        else if (dist != 1 && game.countBetweenChess(now_location, sec_target_location) != 1) {
            System.out.println("砲/炮只能移動到相鄰空格位置或在上/下/左/右恰好隔一個棋子才能吃子！");
            continue;
        }
        else if (game.eat(now_location, sec_target_location)) break;
        else continue;
    }

    if (dist != 1) {
        System.out.println("只能移動到相鄰空格位置！");
        continue;
    }

    if (!game.board[sec_target_location].isOpened) {
        System.out.println("無法吃掉未翻開的棋子！");
        continue;
    }

    if (game.checkSameSide(now_location, sec_target_location)) { // 代表不同陣營，吃對方棋子
        if (game.eat(now_location, sec_target_location)) break;
        else continue;
    }
}
if (quit == 1) continue;
game.Player1_side = (game.Player1_side == 0) ? 1 : 0;
game.nextTurn();
}
else {
    if (game.Player1_side == -1) { // 遊戲開始第一個人翻牌
        game.Player1_side = game.board[target_location].side;
        System.out.println("玩家 小A 為: " + game.colorSide());
    }
}

```

```
        game.board[target_location].isOpened = true; // 翻開棋子
        System.out.println(game.colorSide() + " 翻開了: " + game.board[target_location].name);
        game.Player1_side = (game.Player1_side == 0) ? 1 : 0;
        game.nextTurn();
    }
}
}
```

AI 使用狀況與心得

- 使用層級：
(層級3) 一開始就使用，搭配局部的自己撰寫
- 概述和 AI 互動的次數與內容：
和 AI 的互動次數大約為 24 次，因為對暗棋規則的不熟悉，先詢問 AI 該怎麼開頭，然後一步一步根據 AI 的意見和課本講義寫本遊戲。
在測試過程中也透過 AI 除錯迅速找到漏洞部分，最後遊戲一定程度完整後，詢問 AI 本遊戲是否還存在漏洞。
- 你手動(沒有用AI)的部份
 - showAllChess
 - checkSameSide
 - colorSide
 - 普通棋子吃棋
 - 主程式遊戲流程

心得

在實作的過程中，AI 最大的幫助在於引導我如何運用 OOP。原本我習慣將所有邏輯塞在 Main 裡面，但透過與 AI 討論，我學會了將棋子的屬性(Chess)與遊戲規則(ChessGame)拆分，並利用抽象類別 AbstractGame 來規範遊戲的基本行為。

而在使用 AI 也幫我省下了一些簡單計算的程式，像是計算棋子間的棋子數、棋子的產生等基礎程式。

但同時 AI 也並不是完全正確的，當我們下達的指令不夠嚴謹明確的話，就可能會給我們一些不太正確的回覆，在寫棋子的選擇以及移動或吃子目標位置時，AI 所提供參考的程式就有點錯亂，導致棋子能隨意移動等情況發生，所以只能親自在經過反覆測試和修改才能讓本遊戲有序地進行。

透過此次作業，讓我更加了解物件導向，更加熟悉 Java 的基礎使用。